

DEV_IMPL_GUIDE - SUB-WF-PAUSE-01 Subscription Pause

1. Purpose

This guide explains pause implementation as the flow executes at runtime.

Use it with:

- DD_SUB-WF-PAUSE-01-Subscription_Pause-v1.0.md
- DD_SUB-WF-PAUSE-01-FLOW-WIK-v1.0.md
- DD_SUB-WF-FRAMEWORK-01-Subscription_Workflow_Framework-v1.0.md
- DD_SUB-WF-FRAMEWORK-01-WORKERS-v1.0.md

2. Runtime Actors

Runtime component	What it does
subscription-workflow-api pod	Receives pause request, validates, starts operation.
SUB-LM module/API	Provides authoritative subscription status and mutation APIs.
BIL-01	Computes pause intent and fee.
FUL-04	Suspends service/restriction fulfillment.
Camunda	Runs pause BPMN.
Drools	Validates pause policy and duration.
Worker apps	Execute BPMN service task topics.

3. Before The Request Can Work

Required:

- pause_reason seeded.
- subscription_pause_config seeded.
- subscription_pause_history migrated.
- subscription_operation_config has (WIK, PAUSE) -> sub-pause-wik.
- Drools package rules.subscription.pause.wik.
- BPMN process sub-pause-wik.

4. Step 1 - Caller Sends Pause Request

Endpoint:

```
POST /api/subscriptions/sub_01HZ_KAMAU_FIBER/pause HTTP/1.1
Authorization: Bearer <jwt-or-service-token>
Idempotency-Key: pause-sub_01HZ_KAMAU-vacation-20260315
Content-Type: application/json

{
  "operatorCode": "WIK",
  "pauseTrigger": "CUSTOMER_REQUESTED_PAUSE",
  "pauseReasonCode": "CUSTOMER_VACATION",
  "durationDays": 31,
  "notes": "Customer travelling to Mombasa for holiday season",
  "initiatingWorkflowKind": "BO_UI",
  "initiatingWorkflowRef": "cc-ticket-2026-03-15-3301"
}
```

5. Step 2 - Request Lands On API Pod

The API validates shape:

Field	Validation
operatorCode	Required.
pauseTrigger	CUSTOMER_REQUESTED_PAUSE or ADMIN_PAUSE.
pauseReasonCode	Required.
openEnded, durationDays, resumeScheduledAt	At most one supplied.
durationDays	Positive integer when supplied.
resumeScheduledAt	Future timestamp when supplied.

Actor user/role comes from auth context, not request body.

6. Step 3 - API Loads Required Data

6.1 Subscription Summary

Load from SUB-LM source of truth:

Deployment	How
Same app	SubscriptionReadService.getSummary(...).
Split service	GET /internal/subscriptions/{id}/summary.

Do not use cache for current subscription status.

Expected:

```
{
  "subscriptionId": "sub_01HZ_KAMAU_FIBER",
  "operatorCode": "WIK",
  "customerId": "cust_01HZ_KAMAU",
  "statusCode": "ACTIVE",
  "activeRestrictions": []
}
```

6.2 Pause Config

```
SELECT *
FROM subscription_pause_config
WHERE operator_code = 'WIK';
```

6.3 Pause Reason

```
SELECT *
FROM pause_reason
WHERE code = 'CUSTOMER_VACATION'
AND is_active = TRUE;
```

6.4 Open Pause Check

```
SELECT *
FROM subscription_pause_history
WHERE subscription_id = :subscriptionId
AND actual_resume_at IS NULL;
```

If row exists, return 409 ALREADY_PAUSED.

7. Step 4 - API Calls Drools

Package:

```
rules.subscription.pause.wik
```

Facts:

```
{
  "operatorCode": "WIK",
  "subscription": {
    "subscriptionId": "sub_01HZ_KAMAU_FIBER",
    "statusCode": "ACTIVE"
  },
  "request": {
    "pauseTrigger": "CUSTOMER_REQUESTED_PAUSE",
    "pauseReasonCode": "CUSTOMER_VACATION",
```

```

        "durationDays": 31,
        "openEnded": false,
        "resumeScheduledAt": null
    },
    "reason": {
        "code": "CUSTOMER_VACATION",
        "customerSelfServiceEligible": true,
        "adminOnly": false,
        "maxDurationDays": 90,
        "openEndedAllowed": false,
        "pausesBillCycle": true,
        "suspendsFulfillment": true
    },
    "config": {
        "customerSelfServiceEnabled": true,
        "scheduledPauseEnabled": true,
        "maxFutureScheduledResumeDays": 90,
        "minPauseHours": 24
    },
    "actor": {
        "userId": "cc-agent-wik-007",
        "role": "CALL_CENTER"
    },
    "runtime": {
        "now": "2026-03-15T10:00:00Z"
    }
}

```

Expected decision:

```

{
  "eligible": true,
  "durationMode": "FIXED_DAYS",
  "durationDays": 31,
  "resumeScheduledAt": "2026-04-15T10:00:00Z",
  "pauseFeeApplicable": false,
  "reasonCatalogSnapshot": {
    "code": "CUSTOMER_VACATION",
    "pauses_bill_cycle": true,
    "suspends_fulfillment": true
  }
}

```

8. Drools Rules To Configure

8.1 Subscription Must Be ACTIVE

```

rule "pause requires active subscription"
when
    $facts : PauseFacts(subscription.statusCode != "ACTIVE")
    $decision : PauseDecision()
then
    $decision.reject("INVALID_STATE_FOR_PAUSE", "Pause requires ACTIVE subscription.");
end

```

8.2 Role Trigger Gating

```

rule "customer pause allowed roles"
when
    $facts : PauseFacts(
        request.pauseTrigger == "CUSTOMER_REQUESTED_PAUSE",
        actor.role not in ("CUSTOMER_SELF_SERVICE", "CALL_CENTER", "SUBSCRIPTION_OPERATOR")
    )
    $decision : PauseDecision()
then
    $decision.reject("ROLE_NOT_AUTHORIZED_FOR_TRIGGER", "Role cannot perform customer-requested pause.");
end

rule "admin pause allowed roles"
when
    $facts : PauseFacts(
        request.pauseTrigger == "ADMIN_PAUSE",
        actor.role not in ("SUBSCRIPTION_ADMIN", "SUBSCRIPTION_OPERATOR")
    )
    $decision : PauseDecision()
then
    $decision.reject("ROLE_NOT_AUTHORIZED_FOR_TRIGGER", "Role cannot perform admin pause.");
end

```

8.3 Reason Eligibility

```

rule "pause reason must be active"
when
    $facts : PauseFacts(reason == null || reason.isActive == false)
    $decision : PauseDecision()
then
    $decision.reject("UNKNOWN_OR_INACTIVE_PAUSE_REASON", "Pause reason is unknown or inactive.");
end

rule "self service reason eligibility"
when
    $facts : PauseFacts(
        actor.role == "CUSTOMER_SELF_SERVICE",
        reason.customerSelfServiceEligible == false
    )
    $decision : PauseDecision()
then
    $decision.reject("PAUSE_REASON_NOT_CUSTOMER_SELF_SERVICE", "Reason cannot be used by customer self-service.");
end

```

8.4 Duration Resolution

```

rule "fixed days duration"
when
    $facts : PauseFacts(request.durationDays != null)
    $decision : PauseDecision(rejected == false)
then
    $decision.setDurationMode("FIXED_DAYS");
    $decision.setDurationDays($facts.request.durationDays);
    $decision.setResumeScheduledAt($facts.runtime.now.plusDays($facts.request.durationDays));
end

rule "open ended not allowed"
when
    $facts : PauseFacts(
        request.openEnded == true,
        reason.openEndedAllowed == false
    )
    $decision : PauseDecision()
then
    $decision.reject("OPEN_ENDED_NOT_ALLOWED_FOR_REASON", "Open-ended pause is not allowed for this reason.");
end

rule "duration exceeds reason max"
when
    $facts : PauseFacts(
        request.durationDays != null,
        reason.maxDurationDays != null,
        request.durationDays > reason.maxDurationDays
    )
    $decision : PauseDecision()
then
    $decision.reject("DURATION_EXCEEDS_REASON_MAX", "Requested duration exceeds reason maximum.");
end

```

9. Step 5 - API Starts Operation

After API validation and Drools approval, the pause API asks the shared SUB-WF framework to create the operation and start Camunda.

If Pause and the framework are in the same application, this is an in-process method call, for example:

```
OperationStartService.start(StartOperationCommand command)
```

If the framework is deployed as a separate service, this can be an internal HTTP call, for example:

```
POST /internal/subscription-operations/start
Authorization: Bearer <subscription-workflow-service-token>
```

The payload below is the command/request body passed to that framework start contract. It is not the public customer/admin pause API request.

```

{
    "operatorCode": "WIK",
    "subscriptionId": "sub_01HZ_KAMAU_FIBER",
    "operationKind": "PAUSE",
    "idempotencyKey": "pause-sub_01HZ_KAMAU-vacation-20260315",
    "initiatingActorUserId": "cc-agent-wik-007",
    "initiatingActorRole": "CALL_CENTER",
    "bpmnVariables": {
        "pauseTrigger": "CUSTOMER_REQUESTED_PAUSE",
        "pauseReasonCode": "CUSTOMER_VACATION",

```

```

        "durationMode": "FIXED_DAYS",
        "durationDays": 31,
        "resumeScheduledAt": "2026-04-15T10:00:00Z",
        "reasonCatalogSnapshot": {
            "code": "CUSTOMER_VACATION",
            "pauses_bill_cycle": true,
            "suspends_fulfillment": true
        },
        "pauseFeeApplicable": false
    }
}

```

Framework starts Camunda process:

```
sub-pause-wik
```

10. Step 6 - API Returns Accepted

```

{
  "operationId": "subop_01HZ_KAMAU_PAUSE",
  "operationKind": "PAUSE",
  "bpmnProcessKey": "sub-pause-wik",
  "currentState": "INITIATED",
  "resolvedResumeScheduledAt": "2026-04-15T10:00:00Z"
}

```

11. Step 7 - BPMN Executes Worker Topics

Order	BPMN service task does this	Worker topic/message
1	Validate/route policy.	drools-decide
2	Put subscription in pending pause.	sub-lm-put-pending-status
3	Compute cycle window.	sub-lm-compute-cycle-window
4	Ask billing for pause intent.	bil01-emit-intent
5	Optional pay-first fee.	bil01-charge-pre-authorize then payment-confirmed then bil01-confirm-charge
6	Suspend service.	ful-call-restrict
7	Wait for fulfillment result.	fulfillment-completed or fulfillment-failed
8	Clear active restrictions.	sub-lm-put-active-restrictions
9	Insert pause history row.	sub-lm-put-master-fields
10	Commit subscription suspended.	sub-lm-commit-state
11	Emit pause event.	kafka-emit
12	Notify customer.	notification-send
13	Finalize operation.	subscription-operation-finalize

12. Step 8 - Success Event Is Emitted

Event:

```
SubscriptionPaused
```

Topic:

```
sophix.subscription.subscription.paused
```

Emitted only after SUB-LM commits SUSPENDED.

13. Compensation Flows

Failure	BPMN compensation
Validation reject before pending	kafka-emit SubscriptionPauseRejected, finalize failed.
Billing intent/fee failure	sub-lm-revert-to-prior, emit rejected, finalize failed.
Payment timeout	bil01-revert-pending-charge, sub-lm-revert-to-prior, emit rejected, finalize failed.

Failure	BPMN compensation
FUL-04 failed	bil01-revert-pending-charge if needed, sub-lm-revert-to-prior, emit rejected, finalize failed.
History insert/commit failed after network suspension	Manual recovery.
Notification failed	Log and continue.

14. Manual Smoke Test

1. Seed pause_reason.
2. Seed subscription_pause_config.
3. Seed operation config (WIK, PAUSE).
4. Create active subscription.
5. Send pause API request.
6. Confirm operation row exists.
7. Confirm BPMN process exists.
8. Confirm subscription moves to PENDING_PAUSE.
9. Simulate fulfillment completed.
10. Confirm pause history row inserted.
11. Confirm subscription becomes SUSPENDED.
12. Confirm SubscriptionPaused outbox row exists.
13. Confirm operation final state is COMPLETED.

15. Common Mistakes

- Treating pause and SUSPEND-NP as identical.
- Not snapshotting pause reason.
- Allowing multiple open pause rows.
- Letting customer self-service use admin-only reasons.
- Emitting success before SUB-LM commits SUSPENDED.
- Forgetting scheduled resume scanner dependency.